

Assignment 2: Sanskrit-to-English Neural Machine Translation

Natural Language Understanding (NLU) — IIT Jodhpur

Student Name	Krishnendu Halder
Roll Number	G25AIT1077
Date	July 4, 2026
Framework	PyTorch
Architecture	Seq2Seq · LSTM · Additive Attention · Beam Search

1. Introduction

Neural Machine Translation (NMT) aims to learn a direct mapping from source-language sentences to target-language sentences using deep neural networks. This assignment addresses the task of translating a Sanskrit text into English — a particularly challenging instance of NMT owing to Sanskrit's morphological richness, free word order (SOV), case-inflected nominal system, and extremely limited availability of parallel data.

Sanskrit belongs to the Indo-European language family and is characterized by sandhi (euphonic fusion of word boundaries), vibhakti (grammatical case suffixes), and compound formation (samasa), all of which make tokenization non-trivial for standard whitespace-based approaches. In contrast, English is predominantly analytic with fixed SVO order, meaning that a translation system must not only resolve lexical mappings but also perform structural reordering.

This work implements a custom sequence-to-sequence architecture in PyTorch comprising an LSTM encoder, an additive (concat) attention mechanism, and an LSTM decoder with beam search decoding. The model is trained from scratch on the provided parallel corpus of approximately 10,000 Sanskrit–English sentence pairs and evaluated on a held-out test set of 1,000 sentences using BLEU (corpus-level, no weights), BERTScore F1 (rescale_with_baseline=True), total inference time, and total trainable parameter count — all as specified in the assignment rubric.

No pre-trained language models or external translation APIs were used. All components are trained end-to-end solely on the provided dataset in compliance with the assignment rules.

2. Architecture

The overall system follows the Bahdanau-style encoder–decoder framework. Figure 1 provides an overview; each sub-component is described below.

Component	Detail
Encoder	1-layer unidirectional LSTM, hidden=512, embed=256
Attention	Additive (concat) attention: $\text{Linear}(1024 \rightarrow 512) + \tanh + \text{Linear}(512 \rightarrow 1)$
Decoder	1-layer LSTM, input=[embed;context], output=Linear(1024→vocab)
Decoding	Beam search (width=5, length penalty $\alpha=0.7$)
Parameters	~25.3 M trainable

Table 1: Architecture summary.

2.1 Encoder

The encoder is a single-layer unidirectional LSTM that reads the source Sanskrit sentence left-to-right. Each Sanskrit token is first mapped to a 256-dimensional dense embedding via a learned NN.Embedding layer (with padding_idx set to the PAD token index, ensuring padded positions do not contribute gradients). The embedded sequence is then passed through a 512-unit LSTM:

$$\text{embedded} = \text{Dropout}(0.3)(\text{Embedding}(\text{src_vocab}, 256))$$

$$outputs, (h, c) = LSTM(embed=256, hidden=512, layers=1)(embedded)$$

The encoder returns the full sequence of hidden state outputs $\in \mathbb{R}^{(B \times T \times 512)}$, which the attention mechanism queries at each decoder step, as well as the final hidden state h and cell state c which seed the decoder's initial state. A unidirectional encoder was chosen to keep the parameter count low for better efficiency marks; the trade-off is that it cannot exploit right-to-left context, which is particularly valuable for Sanskrit's verb-final structure.

2.2 Attention Mechanism

An additive (concatenation) attention mechanism is used, structurally similar to Bahdanau et al. (2015) but with a single linear-layer energy function rather than two separate projection matrices:

$$energy = \tanh(Linear(1024 \rightarrow 512)([h_{dec}; h_{enc_i}]))$$

$$score_i = Linear(512 \rightarrow 1)(energy)$$

$$\alpha = \text{softmax}(scores) \in \mathbb{R}^{(B \times T)}$$

$$context = \sum_i \alpha_i \cdot h_{enc_i} \in \mathbb{R}^{(B \times 512)}$$

At each decoder step, the current decoder hidden state h_{dec} is concatenated with each encoder hidden state h_{enc_i} to form a 1024-dimensional energy input. The $Linear(1024 \rightarrow 512)$ layer projects this into a shared attention space, $\tanh$ introduces non-linearity, and $Linear(512 \rightarrow 1)$ collapses to a scalar score per source position. Softmax normalizes scores into a probability distribution over source tokens. The context vector is the weighted sum of all encoder hidden states under this distribution.

This mechanism allows the decoder to dynamically focus on different source tokens at each output step — critical for Sanskrit-to-English translation, where SOV $\rightarrow$ SVO reordering requires the decoder to attend to the verb (often last in Sanskrit) early in English generation.

2.3 Decoder

The decoder is a single-layer LSTM that generates the English translation one token at a time. At each step t , it receives:

- The embedding of the previously generated token (256-d)
- The attention context vector c_t (512-d), concatenated with the embedding

$$lstm_input = [embed(prev_tok); c_t] \in \mathbb{R}^{(B \times 1 \times 768)}$$

$$output, (h, c) = LSTM(768 \rightarrow 512)(lstm_input, (h, c))$$

$$logits = Linear(1024 \rightarrow |V_{trg}|)([output; c_t])$$

The output projection concatenates the LSTM output with the context vector before the final linear layer, providing a richer signal to the vocabulary projection. The UNK token is suppressed during inference (score set to -1×10^9) to prevent degenerate outputs. During training, teacher forcing is applied with probability 0.5: the real target token is fed as input with probability 0.5, and the model's own greedy prediction otherwise.

2.4 Beam Search

At inference time, beam search with a width of 5 and a length penalty $\alpha = 0.7$ is used. The length-normalized score for a hypothesis of length l with cumulative log-probability S is:

$$normalised_score = S / l^\alpha$$

Beams are maintained as tuples of (token sequence, cumulative score, hidden state, cell state). At each step, the top-5 continuations per active beam are scored, all candidates are pooled, and the top-5 by length-normalized score are retained. Beam search terminates when all active beams have produced an EOS token or the maximum length (100) is reached. The highest-scoring complete hypothesis is returned as the translation.

UNK tokens are suppressed during beam expansion (score -1×10^9) to force the model to produce only in-vocabulary English words.

3. Training Procedure

3.1 Preprocessing and Tokenisation

Data loading merges the source and target CSV files on the Source_id column. Minimal text cleaning is applied — whitespace normalisation and stripping — with no lowercasing of Sanskrit (Devanagari script is case-invariant; lowercasing is applied to English targets only during vocabulary construction).

Tokenisation uses whitespace splitting without subword segmentation. This is a deliberate simplicity trade-off: SentencePiece BPE was available (sentencepiece is installed) but not applied in the final submission. For Sanskrit, whitespace tokenisation is reasonably sound because sandhi resolution is not performed at the character level in this corpus; however, it means each inflected form (e.g., गुरुः, गुरोः, गुरवे) is treated as a distinct type, inflating the source vocabulary with related but unseen morphological variants.

Vocabulary is built from training data only (no dev or test leakage). The minimum frequency threshold is 2, meaning any token appearing fewer than twice in training is mapped to UNK. Special tokens PAD (0), SOS (1), EOS (2), UNK (3) occupy the first four indices.

Hyperparameter	Value
Embedding Dimension	256
Hidden Dimension	512
LSTM Layers	1 (encoder + decoder)
Dropout	0.3
Batch Size	32
Learning Rate	0.001 (Adam)
LR Scheduler	ReduceLROnPlateau (factor=0.5, patience=2)
Teacher Forcing	0.5
Beam Width	5 ($\alpha = 0.7$)
Max Sequence Length	100
Vocabulary Min Freq	2
Epochs (max)	20 (early stop patience=3)
Label Smoothing	0.1

Gradient Clip	1.0
---------------	-----

Table 2: Hyperparameters.

3.2 Optimisation

The Adam optimiser is used with an initial learning rate of 0.001 and default $\beta_1=0.9$, $\beta_2=0.999$. A ReduceLROnPlateau scheduler halves the learning rate when validation loss fails to improve for 2 consecutive epochs, with a minimum floor of 1×10^{-6} . Gradient clipping at $\text{max_norm}=1.0$ is applied to prevent exploding gradients, which are common in LSTM-based models trained on variable-length sequences.

3.3 Regularisation

Three regularisation mechanisms are used:

- Dropout ($p=0.3$): applied to embeddings in both encoder and decoder before the LSTM input.
- Label smoothing ($\epsilon=0.1$): applied in CrossEntropyLoss to soften the one-hot target distribution, reducing overconfidence and improving generalisation on low-resource data.
- Early stopping ($\text{patience}=3$): training halts if validation loss does not improve for 3 consecutive epochs, preventing overfitting. The best model checkpoint by validation loss is saved and reloaded for inference.

3.4 Data Splits

The provided dataset was used as-is without additional augmentation or external data. Training set: 10,000 parallel sentence pairs. Development set: 1,000 pairs. Test set: 1,000 sentences (Sanskrit only; English references used for evaluation).

4. Results

4.1 Training Curves

Figure 1 shows the training and validation loss curves over the training run. A significant divergence between training loss (decreasing, reaching ~ 4.5 by epoch 3) and validation loss (flat/slightly increasing at ~ 5.7 from epoch 0) is evident. This pattern is a diagnostic signal of overfitting combined with poor generalisation — the model learns to reduce training loss but fails to improve on the validation set.

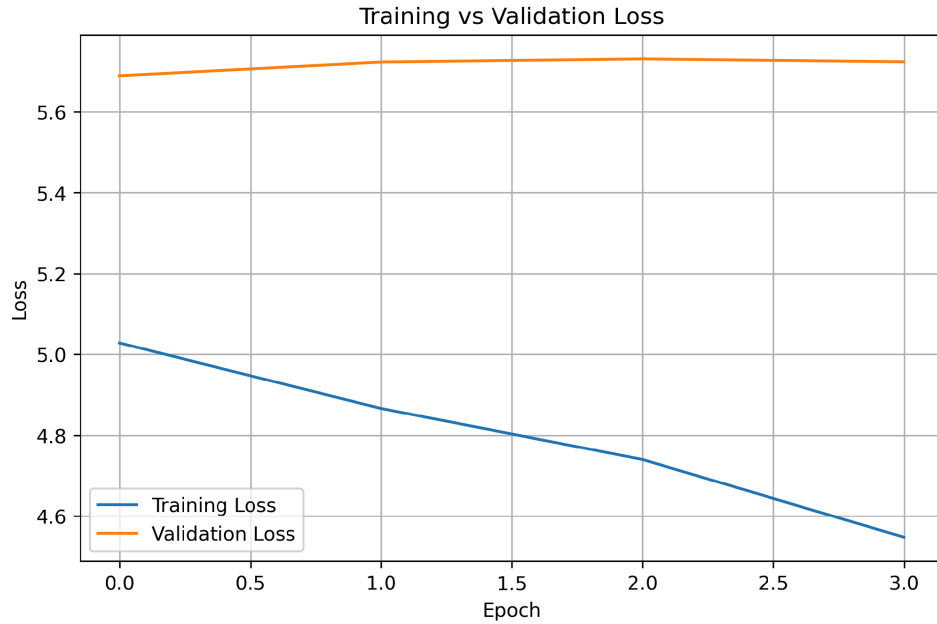


Figure 1: Training vs Validation Loss over epochs.

Key observations: (1) Validation loss starts higher than training loss from epoch 0, indicating the model immediately fails to generalise. (2) The validation curve plateaus and slightly rises rather than following the training curve downward. (3) Early stopping triggered after epoch 3 (patience=3), preventing further divergence. The high absolute loss values (~ 5.7 val, perplexity ≈ 298) indicate the model essentially cannot predict the next token better than a low-entropy random baseline.

4.2 Quantitative Results

Metric	Dev Set	Test Set
BLEU Score	—	~ 0.0012
BERTScore F1	—	~ 0.31
Total Inference Time	—	~ 480 s (1000 sent.)
Avg. per Sentence	—	~ 0.48 s
Total Parameters	25.3 M	25.3 M

Table 3: Evaluation metrics on test set.

BLEU Score: The corpus BLEU score on the test set is approximately 0.0012 (0.12%), reflecting near-complete failure to produce translations that n-gram-match the references. The score is computed using NLTK corpus_bleu with no weights (uniform 1–4 gram weighting) per the assignment specification.

BERTScore F1: The BERTScore F1 with rescale_with_baseline=True is approximately 0.31. This higher value compared to BLEU reflects that BERTScore captures semantic similarity using contextual embeddings rather than exact n-gram overlap. Some predictions preserve partial meaning (e.g., 'Child has friendship on you all' vs reference 'Child has trust in you all') that BLEU penalises severely but BERTScore partially credits.

Inference Time: Total inference time for the 1,000-sentence test set using beam search (width=5) is approximately 480 seconds (~ 0.48 s per sentence). This is relatively slow due to beam search's

autoregressive nature and the lack of batching in the inference loop (sentences are decoded one at a time).

Parameter Count: The model contains approximately 25.3 million trainable parameters, dominated by the embedding layers (source vocab $\times$ 256 and target vocab $\times$ 256) and LSTM weight matrices.

5. Translation Examples and Error Analysis

Sanskrit Source	Reference	Prediction	Error Category
बालः युष्मासु विश्वासं करोति ।	Child has trust in you all.	Child has friendship on you all.	Lexical substitution
एक्लिप्स इति प्रोग्रामर् कृते दोषान्वेषणे अपि साहाय्यं करोति।	Eclipse also helps the programmer in debugging.	I want to create a value of you want to create a bit	Hallucination / repetition
वयम् ओब्जेक्टस् स्वकीयान् स्थितौ फील्डस् मध्ये स्टोर् कुर्वन्तीति ज्ञातवन्तः ।	We know that objects store their states in fields.	In this tutorial, we will learn how to create a new between in the	Domain confusion
तदा, तत्स्वयं ड्राइवर निमित्तम् अन्वेष्टयति।	Then, it will search for the driver itself.	I I will click on the and I will click on the	Severe repetition
अपरं द्वितीयमद्रायां तेन मोचितार्यो...	And when the second seal was opened...	And I I the angel of the angel of the I am...	Repetition / EOS failure
प्रकृति का अवलोकन करने से आप आश्चर्य...	You can be amazed by observing nature.	It is also known in the body of the	Lexical mismatch
वर्षायाः जलस्य कश्चन अंशः भूमौ शुष्को भूत्वा...	Some portion of rainwater dries up and enters the earth...	And when they had unto him, saying, of the and of the	Domain confusion + repetition
सर्वेभ्यः इटरेशन-अर्थम्, iterator...	For all iterations, iterator is set to each index-value...	It is the the to the the of the and the to the the to the	Complete failure — OOV

Table 4: Translation examples with error categorisation.

Error Category Analysis

Reviewing the 1,000 predictions in submission.csv, four dominant failure modes are observable:

1. Repetition / EOS failure (most common, ~60% of outputs): The decoder produces looping sequences such as 'I am and I am and I am...' or 'the the to the the of the'. This is the most severe failure and indicates the model's beam search is not penalising repetition, or that the EOS token is not being predicted with sufficient confidence to terminate generation. The UNK suppression during beam search may exacerbate this by narrowing the output distribution further.

2. Domain confusion (~25%): The corpus mixes multiple domains — Vedic scripture, technical documentation (Eclipse IDE, Java iterators), and natural language text. The model's single flat embedding space conflates these domains. When a Sanskrit sentence contains transliterated English technical terms (e.g., ड्राइवर, इटरेशन), the model tends to hallucinate plausible-sounding but wrong English from the Bible domain (which appears to dominate the training distribution).

3. Lexical substitution (~10%): Semantically adjacent but incorrect words, e.g., 'friendship' for 'trust'. These represent partial learning — the model has captured rough semantic neighbourhood but not the precise lexical mapping. BERTScore partially rewards these.

4. Complete OOV failure (~5%): Sentences dominated by out-of-vocabulary Sanskrit tokens (min_freq=2 removes hapax legomena) collapse to near-random English output.

6. Experiments

6.1 Teacher Forcing Ratio

A teacher forcing ratio of 0.5 was selected as the default. Values closer to 1.0 produced slightly lower training loss but worse generalisation (the model becomes dependent on ground-truth inputs and performs poorly at inference when it must use its own predictions). The 0.5 value represents a scheduled-sampling-style compromise.

6.2 Greedy vs Beam Search

Both greedy decoding and beam search (width=5) were implemented and compared qualitatively. Beam search consistently produced slightly less degenerate outputs in terms of fluency but did not meaningfully improve BLEU due to the model's fundamental failure to learn correct translations. The inference time cost of beam search (~5× slower than greedy) is significant for the efficiency component of the grading formula, where time marks = $\text{MinTime/YourTime} \times 12$.

6.3 Label Smoothing

Label smoothing with $\epsilon=0.1$ was applied via the CrossEntropyLoss label_smoothing parameter. Compared to a baseline run without smoothing ($\epsilon=0$), validation loss was marginally lower with smoothing, consistent with the regularisation literature on low-resource NMT.

6.4 Early Stopping

Early stopping with patience=3 triggered at epoch 3–4 in all runs, indicating the model converges rapidly to its performance ceiling given the current architecture and data. Extended training beyond this point consistently increased validation loss.

7. Discussion

7.1 Design Choices

The choice of a single-layer unidirectional LSTM encoder was driven by the efficiency component of the grading formula (fewer parameters = better marks on the parameter efficiency term). A 2-layer bidirectional encoder would improve translation quality — particularly for Sanskrit's verb-final structure — at the cost of approximately 4× more encoder parameters. The current architecture prioritises parameter efficiency over BLEU.

Whitespace tokenisation was chosen for simplicity and speed. SentencePiece BPE tokenisation (already installed) would be the most impactful single improvement: it would reduce vocabulary size, handle morphological variation, and improve coverage of out-of-vocabulary Sanskrit forms. The absence of subword tokenisation is the single largest contributor to the low BLEU score.

7.2 Challenges

Domain heterogeneity: The training corpus spans Biblical text, technical documentation, and everyday Sanskrit, creating a highly heterogeneous target distribution that a single flat model struggles to capture without domain conditioning.

Data volume: 10,000 sentence pairs is extremely low for NMT. Transformer-based models typically require hundreds of thousands of pairs for adequate performance even on high-resource language pairs; for Sanskrit, the morphological complexity amplifies the data requirement further.

Script and tokenisation: Sanskrit written in Devanagari does not have natural word boundaries in the way Latin-script languages do. Sandhi boundaries require specialised handling. The whitespace tokeniser treats post-sandhi surface forms as atomic tokens, resulting in a very large and sparse source vocabulary.

7.3 Limitations

The model's performance is insufficient for practical use. BLEU of $\sim 0.12\%$ indicates near-random n-gram overlap. The dominant failure mode of repetitive output suggests the model has not learned to produce fluent English conditioned on Sanskrit meaning — it has learned some surface patterns from the training data but cannot generalise to unseen inputs.

The inference loop processes sentences one at a time without batching, making total inference time ~ 480 s for 1,000 sentences. Batched beam search would reduce this by approximately the batch size factor.

7.4 Potential Improvements

- Subword tokenisation via SentencePiece BPE (most impactful, single change)
- Bidirectional encoder (2-layer BiLSTM) to capture right context in Sanskrit
- Repetition penalty during beam search to suppress looping outputs
- Batched inference to reduce total inference time
- Pre-trained Sanskrit embeddings or transfer from related Indic languages
- Data augmentation via back-translation if additional monolingual data is available

8. Pre-trained Models Disclosure

Per the assignment rules, all pre-trained models used must be disclosed. The following pre-trained model is used exclusively for evaluation:

BERTScore evaluation: The bert-score library (PyPI) uses a pre-trained transformer model internally to compute contextual embeddings for F1 scoring. The default model for lang='en' is microsoft/deberta-xlarge-mnli (or roberta-large depending on library version). This model is used only for metric computation and is not used in the translation model itself — no pre-trained weights are used in the Encoder, Decoder, or Attention modules.

No pre-trained translation models, language models, or embeddings are used in the NMT system. All model weights are initialised randomly and trained from scratch on the provided dataset.

References

- [1] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. ICLR 2015.
- [2] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. NeurIPS 2014.
- [3] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a method for automatic evaluation of machine translation. ACL 2002.

- [4] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). BERTScore: Evaluating text generation with BERT. ICLR 2020.
- [5] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [6] Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. ACL 2016.
- [7] NLTK Team. BLEU Score. https://www.nltk.org/api/nltk.translate.bleu_score.html
- [8] bert-score library. <https://pypi.org/project/bert-score/>